

Compresión PNG

Servicios multimedia



Grupo de Tecnología Electrónica
y Comunicaciones



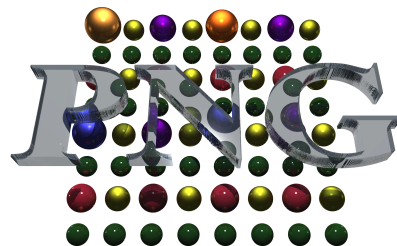
UNIVERSIDADE DA CORUÑA

UNIVERSIDADE DA CORUÑA

Origen de PNG

El problema con GIF

- PNG (Portable Network Graphics) nació a mediados de los 90 como respuesta al problema con el formato GIF.
- El algoritmo de compresión de GIF (LZW) estaba patentado por Unisys.
- Cualquier software que creara o leyera archivos GIF teóricamente tenía que pagar regalías (*royalties*).
- Esta situación era insostenible para la creciente comunidad de software de código abierto y para la web.



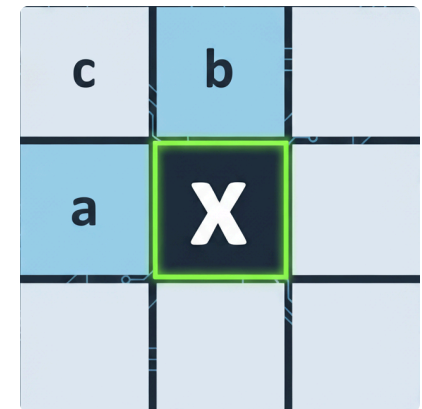
- Sin patentes: Totalmente libre de usar, implementar y distribuir.
- Sin pérdidas (Lossless): A diferencia de JPEG, preserva cada píxel exactamente. Ideal para logotipos y gráficos.
- Estandarización (1996): PNG se estandarizó en 1996 (RFC 2083).

- PNG soporta varios modos de transparencia, a diferencia de GIF que solo soporta transparencia binaria (1 bit).
- Canal alfa completo (RGBA): cada pixel puede tener hasta 256 niveles de transparencia.
 - 0: Transparente
 - 255: Opaco
- Transparencia indexada: se usa una paleta de colores que pueden tener transparencia.
 - Se ahorra espacio en el archivo al no almacenar los colores, solo los índices.
- *Color keying*: un único color se marca como transparente.

- El funcionamiento de la compresión PNG se basa en dos pasos:
 - 1: Pre-compresión (filtrado)
 - En lugar de comprimir píxeles en bruto, PNG primero transforma los datos buscando similitudes.
 - Predice un valor basado en píxeles vecinos y guarda la diferencia ("error").
 - 2: Compresión (DEFLATE)
 - Los datos filtrados (más repetitivos) se comprimen usando *DEFLATE* (el mismo algoritmo que usan los archivos ZIP, una mezcla de LZ77 y Huffman).

- Una imagen se procesa fila por fila (*scanlines*).
- El codificador solo mantiene en memoria la fila actual y la anterior.
- Ejemplo: 3 píxeles RGB
 - Píxeles originales:
 - (R:50, G:100, B:150)
 - (R:52, G:105, B:150)
 - (R:55, G:110, B:151)
 - Scanline:
[50, 100, 150, 52, 105, 150, 55, 110, 151]

- Existen 5 tipos de filtros posibles que se pueden aplicar a cada *scanline*.
- Se prueba cada filtro y se elige el más eficiente.
- El objetivo es minimizar los valores absolutos de las diferencias.
- Los filtros se basan en los valores de los píxeles vecinos:
 - **x**: píxel actual
 - **a**: píxel a la izquierda
 - **b**: píxel arriba
 - **c**: píxel diagonal superior izquierdo



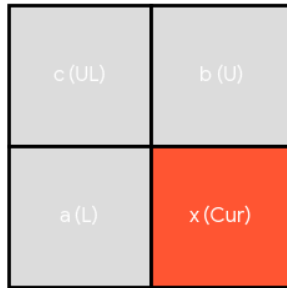
- Para aplicar el filtro, se deben comparar las componentes de color iguales de cada píxel (R con R, G con G, B con B).
- Los *bpp* (bytes por píxel) permiten determinar qué bytes se deben comparar.
- Con 8 bits por componente, $\text{bpp} = 3$.

- Ej: Suponemos dos píxeles, 8 bits por componente (`bpp = 3`):
 - (R:50, G:20, B:150)
 - (R:32, G:40, B:19)
- bytes:
 - [50, 20, 150, 32, 40, 19]
- Para comparar el segundo píxel con el primero:
 - 32 con 50 (posición 3 vs 0)
 - 40 con 20 (posición 4 vs 1)
 - 19 con 150 (posición 5 vs 2)

Proceso de compresión

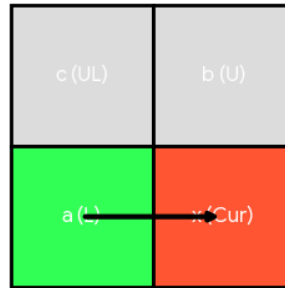
Tipos de filtro

None



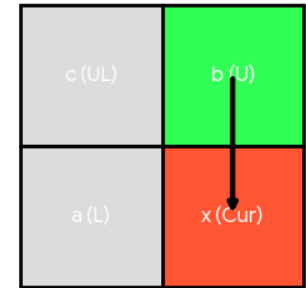
$$\text{Filt}(x) = x$$

Sub



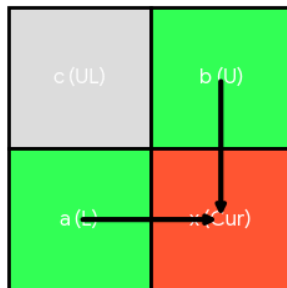
$$\text{Filt}(x) = x - a$$

Up



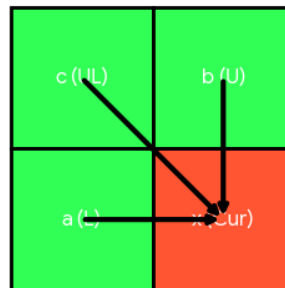
$$\text{Filt}(x) = x - b$$

Average



$$\text{Filt}(x) = x - \text{floor}((a + b) / 2)$$

Paeth



$$\text{Filt}(x) = x - \text{Paeth}(a, b, c)$$

■ Filtro None:

- $\text{Dato_Filtrado} = \text{Valor_Real}$
- Se usa cuando los datos son muy aleatorios.

■ Filtro Sub:

- Se usa el píxel a la izquierda.
- $\text{Dato_Filtrado} = \text{Valor_Real} - a$
- Eficaz para gradientes horizontales.

■ Ejemplo usando filtro Sub:

- Píxeles originales:

- (50, 100, 150)

- (52, 105, 150)

- bytes:

- `[50, 100, 150, 52, 105, 150]`

- Resultado:

- `[50, 100, 150, 2, 5, 0]`

■ Filtro Up:

- Se usa el píxel de arriba.
- $\text{Dato_Filtrado} = \text{Valor_Real} - b$
- Eficaz para gradientes verticales.

■ Filtro Average:

- Se usa el promedio de los píxeles a la izquierda y de arriba.
- $\text{Pred} = \text{floor}((a + b) / 2)$
- $\text{Dato_Filtrado} = \text{Valor_Real} - \text{Pred}$
- Buen filtro general.

■ Filtro Paeth:

- Se usa el predictor de Paeth.
- `Pred = PaethPredictor(a, b, c)`
- `Dato_Filtrado = Valor_Real - Pred`
- Ideal para patrones diagonales o complejos.

```
def PaethPredictor(a, b, c):  
    p = a + b - c  
    pa = abs(p - a)  
    pb = abs(p - b)  
    pc = abs(p - c)  
    return a if pa <= pb and pa <= pc else (b if pb <= pc else c)
```

■ Ejemplo usando filtro Paeth:

▪ Píxeles:

```
100 102  
105 107
```

▪ Para 107: $p = 105 + 102 - 100 = 107$

▪ Predicción = 105

▪ Resultado filtrado: $107 - 105 = 2$

- Todos los *scanlines* filtrados se concatenan.
- DEFLATE = LZ77 + Huffman.
- LZ77 es similar a LZW, pero...
 - Sin un diccionario previo.
 - Reemplaza repeticiones por referencias (distancia, longitud).
 - La distancia se codifica con 15 bits, por lo que la ventana de búsqueda máxima es de 32768 bytes (32 KB).
 - Si no hay coincidencia → se emite un literal.
 - Ejemplo de funcionamiento paso a paso:
<https://valentinbarral.github.io/lz77-demo/>

- Una vez LZ77 produce literales o longitudes y distancias, se codifican con Huffman.
- Los símbolos más frecuentes tienen códigos más cortos.
- En DEFLATE existen dos árboles de Huffman:
 - Uno para literales y longitudes.
 - Otro para distancias.
 - Se hace así porque tienen distinta probabilidad de aparición.
 - Se pueden usar árboles estáticos (se definen en el estándar y no se mandan en el archivo) o dinámicos (se construyen a partir de los datos y se mandan en el archivo).

- Un archivo PNG no es solo los datos comprimidos.
- Está compuesto por *chunks* (bloques estructurados).
- Cada *chunk* tiene:
 - Longitud (4 bytes)
 - Tipo (4 bytes)
 - Datos (podrían ser 0)
 - CRC (4 bytes de verificación)

- Los siguientes chunks son obligatorios:

Chunk	Función
IHDR	Contiene metadatos básicos: ancho, alto, tipo de color, profundidad, método de compresión, filtrado e interlazado.
IDAT	Contiene los datos de imagen comprimidos. Puede haber varios y se concatenan. Aquí también van los arboles dinámicos de Huffman si se usan.
IEND	Marca el final del archivo PNG. Siempre vacío.

■ chunks opcionales:

Chunk	Propósito
PLTE	Define una paleta de colores utilizada como índice. Solo aparece en imágenes con tipo de color indexado.
gAMA	Indica el valor de corrección <i>gamma</i> que debe aplicarse para mostrar la imagen con el brillo correcto en distintas pantallas o dispositivos.
tRNS	Añade transparencia simple sin necesidad de canal alfa completo. Puede definir un color (o índice de paleta) que será totalmente transparente.

■ Mas chunks opcionales:

Chunk	Propósito
tEXt / zTXt / iTXt	Almacenan comentarios o metadatos en texto. – tEXt : texto sin comprimir. – zTXt : texto comprimido. – iTXt : texto en UTF-8 e internacionalizable.
pHYs	Indica la resolución física de la imagen (píxeles por metro). Permite reproducir la imagen con tamaño real al imprimir o escalar.